Aspectos destacados

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez, Ernesto Exposito

- Nuestra arquitectura autonómica anterior detectaba condiciones, pero nunca planificaba ni actuaba.
- Sus alarmas son ráfagas escasas de toda la línea; con ellas ningún planificador recupera retraso.
- Un detector por estación sobre tiempos de proceso hace que reparar en bucle cerrado compense.
- Un LLM de frontera que veta los candidatos de un test estadístico duplica a la mejor regla.
- Un inicio de fallo fijo dentro de la ráfaga de alarmas había producido ganancias engañosas.

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez[a,b,*], Ernesto Exposito[b]

[a] *Universidad Nacional Experimental del Táchira, San Cristóbal, Venezuela*
[b] *Univ Pau & Pays Adour / E2S UPPA, Laboratoire d'informatique de l'Université de Pau et des Pays de l'Adour (LIUPPA), EA3000, 64000 Pau, Francia*

Resumen

Las arquitecturas autonómicas para la Industria 4.0 prometen una producción que se autorrepara, pero los sistemas publicados a menudo se quedan en la detección. Revisamos nuestra propia arquitectura anterior de fabricación autocoordinada, comprobamos que implementa Monitor y Analyze pero no Plan, Execute ni una base de conocimiento, portamos su simulador a un motor sin interfaz gráfica y con pruebas sobre el conjunto de datos de la línea de producción de Bosch, y cerramos el bucle con un espacio de acciones construido sobre las estaciones gemelas paralelas de la línea, un validador simbólico, fallos sintéticos en escalón, intermitentes y de deriva, y un modelo de efectos explícito. Comparamos planificadores basados en reglas con planificadores basados en modelos de lenguaje de gran tamaño (LLM), con y sin conocimiento operativo recuperado, sobre un modelo local de 7B (Qwen2.5) y un modelo de frontera (Claude Fable 5.1); todas las decisiones de diseño se toman en una traza de desarrollo y la evaluación usa 216 escenarios con fallo de tres trazas reservadas, midiendo el retraso recuperado en las 24 h siguientes al fallo. La capa Analyze original es el cuello de botella: en el día siguiente a un fallo nombra a la estación averiada solo en la mitad de los escenarios, pocas veces y siempre dentro de ráfagas de alarmas de toda la línea, y con ella ningún planificador recupera más del 7 % del retraso. Un detector por estación sobre los tiempos de proceso, elegido en

*Autor de correspondencia.
Email addresses: mbsanchez@unet.edu.ve (Manuel Sánchez), ernesto.exposito@univ-pau.fr (Ernesto Exposito)

Versión de lectura en español. El manuscrito que se envía a Computers in Industry es la versión en inglés.

la traza de desarrollo, permite a la mejor regla recuperar el 20 %. Con ese detector, un LLM de frontera que veta los candidatos de un test estadístico de cambio recupera el 41 % (intervalo al 95 %: 32–50 %), por delante de todas las configuraciones de reglas que probamos, incluida una elegida en las propias trazas reservadas, y del mismo diseño sobre el modelo de 7B (29 %); usado sin el test, el mismo modelo no supera a la regla. Un primer diseño de evaluación, con el fallo inyectado en un instante fijo dentro de la ráfaga de arranque del detector, había producido ganancias aparentes que invertían esta clasificación.

Keywords: Computación autonómica, MAPE-K, Industria 4.0, Modelos de lenguaje de gran tamaño, Generación aumentada por recuperación, Gestión de fallos

---

1. Introducción

La computación autonómica propone sistemas que se gestionan a sí mismos mediante un bucle Monitor–Analyze–Plan–Execute sobre una base de conocimiento compartida (MAPE-K) [1, 2]. En la fabricación este bucle resulta atractivo porque las líneas de producción se degradan continuamente y la supervisión humana no escala: una línea que se autorrepara detectaría una estación que se degrada y se reconfiguraría antes de que el retraso se acumule.

En un trabajo anterior propusimos una arquitectura para procesos de fabricación autocoordinados con propiedades self-* y la evaluamos con un simulador, SSACSimulator, sobre el conjunto de datos de la línea de producción de Bosch [3, 4]. Al revisar ese código, comprobamos que solo implementa las fases Monitor y Analyze. Cuando uno de sus tres detectores se dispara, el simulador escribe *"launching the autonomous cycle for reconfiguration"* en un panel de registro; no se produce ningún plan y no se ejecuta nada. El bucle se anunciaba, pero no se cerraba.

Este artículo lo cierra y se pregunta qué hace falta realmente para cerrarlo. Dos desarrollos hacen oportuna la pregunta. Primero, los modelos de lenguaje de gran tamaño (LLM) se proponen cada vez más como planificadores para la toma de decisiones industriales, desde el control de sistemas de producción modulares [5] hasta la planificación y programación avanzadas [6]. Segundo, la generación aumentada por recuperación (RAG) [7] es una candidata natural para el componente Knowledge de MAPE-K: en lugar de una ontología construida a mano, el planificador recupera el historial

operativo que necesita. Si alguno de los dos ayuda, frente a qué baseline y con qué capa Analyze, es una cuestión empírica.

Nuestras contribuciones son:

1. Un port fiel, sin interfaz gráfica y con pruebas del simulador original [3], y un análisis de su capa Analyze sobre los datos publicados: un detector es inalcanzable, otro se ejercita con el 0,004 % de los eventos y el tercero emite ráfagas de alarmas de toda la línea, concentradas al inicio de cada traza, que en el día siguiente a un fallo nombran a la estación averiada solo en la mitad de los escenarios (secciones 3 y 7.1).
2. Las fases Plan, Execute y Knowledge que faltaban, y una capa Analyze por estación sobre los tiempos de proceso: un espacio de acciones basado en las estaciones "gemelas" paralelas de la línea de Bosch, un validador simbólico, inyección de fallos con tres formas y un modelo de efectos explícito y parametrizado (sección 4).
3. Una comparación de planificadores basados en reglas con planificadores LLM con y sin conocimiento recuperado, y con un híbrido en el que un test estadístico propone y el LLM veta, sobre un modelo local de 7B y un modelo de frontera, con las dos capas Analyze; todas las decisiones de diseño se toman en una traza de desarrollo y la evaluación usa 216 escenarios con fallo de tres trazas reservadas (secciones 5–7).
4. Evidencia de que es la capa Analyze, y no el planificador, la que decide si cerrar el bucle recupera producción; de que, con alarmas útiles, un LLM capaz aporta más como veto sobre un test estadístico; y de que un inicio de fallo fijo puede fabricar conclusiones que se invierten con un diseño más sólido (sección 8).

Todo el código, los scripts de procesamiento de datos, los prompts, cada decisión de los modelos con la razón que dieron y todos los resultados están disponibles (sección Apéndice A).

2. Antecedentes y trabajo relacionado

Computación autonómica y sistemas autoadaptativos. El modelo de referencia MAPE-K [1] estructura un sistema autogestionado como un bucle de realimentación en torno a un recurso gestionado. La ingeniería de esos bucles es el objeto de la literatura sobre sistemas autoadaptativos [2], que subraya que la fase Analyze debe separar el cambio real del ruido antes de planificar cualquier adaptación. Recientemente se han estudiado los LLM como componentes de esos bucles: una revisión del estado del arte y hoja

de ruta organiza sus usos potenciales en torno a las funciones de MAPE-K [8, 9]; se han integrado agentes basados en LLM en sistemas multiagente basados en MAPE-K [10]; y MAPER extiende MAPE-K con un componente de razonamiento basado en LLM para manejar situaciones imprevistas en tiempo de ejecución [11]. Nuestro estudio se diferencia en que sitúa un LLM dentro del bucle de un sistema industrial cuyo detector primero heredamos sin cambios y después sustituimos, y en que lo mide frente a baselines sin LLM ajustados, sobre datos reservados.

Propiedades self- en la fabricación.* Nuestro trabajo previo integró sistemas multiagente, orientación a servicios y computación autonómica para la coordinación de procesos en la Industria 4.0 [3, 4, 12]. El presente artículo reexamina empíricamente esa arquitectura en lugar de proponer una nueva.

Gestión de alarmas y detección de cambios. En las industrias de proceso, las normas de gestión de alarmas exigen racionalizar las alarmas, de modo que cada una tenga sentido y requiera una respuesta, antes de pedir a los operadores que actúen sobre ellas [13]. Un controlador autonómico que actúa sobre alarmas se enfrenta al mismo requisito, sin un operador que absorba el ruido. Detectar un cambio en la tasa de un flujo de eventos o en la distribución de una medida es un problema clásico [14]; nuestros planificadores basados en reglas y nuestro detector por estación son instancias sencillas de él.

LLM y recuperación en la toma de decisiones industriales. RAG condiciona un modelo de lenguaje con documentos recuperados para cada consulta [7]. En la fabricación, se han combinado agentes LLM con gemelos digitales para planificar y controlar la producción modular [5] y se han aplicado a la planificación y programación avanzadas [6], y una revisión reciente de los LLM para el diagnóstico de fallos en la fabricación inteligente cubre, entre otras funciones, la fusión de conocimiento industrial con recuperación aumentada [15]. Aquí, el conocimiento recuperado son estadísticas operativas estructuradas, calculadas por código a partir del propio historial del controlador, no documentos; mantenemos el término RAG para el paso de recuperación e indicamos exactamente qué se recupera (sección 5).

3. El sistema original, revisado

3.1. Simulador y datos

SSACSimulator es un simulador de eventos discretos en Qt/C++. Un modelo de proceso, derivado del conjunto de datos Bosch Production Line

Performance [16], describe 52 estaciones en cuatro sublíneas más los nodos de inicio y fin, 285 transiciones (84 de ellas flujos principales), un tiempo de ciclo de 6 420 min y 1 183 165 casos. Cuatro trazas de eventos de 100 000 transiciones cada una reproducen movimientos de piezas registrados; abarcan entre 54 y 168 horas de producción. Son las trazas preparadas para el estudio original [3]; no extrajimos otras nuevas. La producción se registra por lotes: 62 de las 157 horas de la traza 100m–200m no contienen ningún evento.

3.2. Port sin interfaz gráfica

Portamos a Python el núcleo de simulación (unas 1 440 líneas de lógica en C++, sin contar la interfaz gráfica). El port reproduce el comportamiento del original tal como se lee en su código fuente, incluidos detalles que cambian los resultados: umbrales calculados en precisión simple, un límite de carga de 100 000 eventos y una regla permisiva para las filas mal formadas. Un camino rápido avanza solo las piezas que pueden salir del sistema; el bucle original por tick se conserva como oráculo de pruebas, y una prueba de regresión comprueba que ambos producen detecciones, decisiones y tiempos de vida de pieza idénticos. La paridad es con el original tal como se lee en su código fuente, no con el binario Qt compilado. Una ejecución tarda entre 0,2 y 0,4 s.

3.3. Lo que hace realmente la capa Analyze

El original implementa tres detectores: una estación cuyo tiempo medio de transición supera 1,3 veces su referencia; más del 10 % de las estaciones marcadas dentro de un mismo lote de eventos simultáneos; y un tiempo de vida medio de las piezas que supera 1,1 veces el tiempo de ciclo. Solo se dispara el segundo. El detector de fallos predichos depende de una etiqueta de un clasificador que es positiva en 15 de 400 000 eventos en el conjunto de las trazas (0,004 %). El detector global es inalcanzable: el tiempo de vida medio de las piezas (636 min) está un orden de magnitud por debajo de su umbral (7 062 min). Por último, 45 de las 285 transiciones tienen un tiempo de referencia cero, de modo que cualquier duración positiva marca su estación. La sección 7.1 muestra qué ve el único detector activo cuando se inyectan fallos.

Plan, Execute y Knowledge están ausentes. Ese es el punto de partida del resto del artículo.

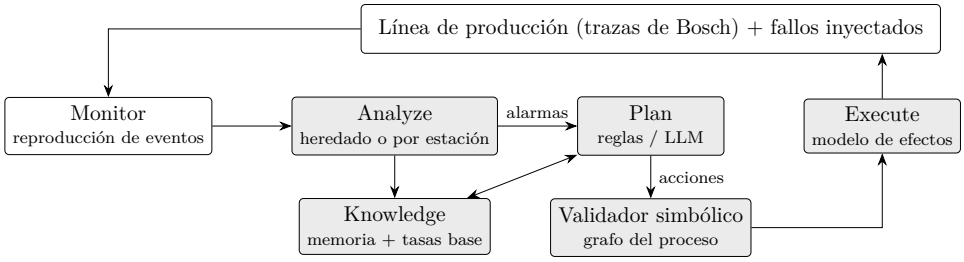


Figura 1: El bucle MAPE-K cerrado. En blanco: presente en el sistema original [3]. En gris: añadido o sustituido en este trabajo. La capa Analyze es el detector heredado o el detector por estación de la sección 4.4.

4. Cerrar el bucle

La figura 1 resume el bucle cerrado. Los componentes añadidos en este trabajo aparecen sombreados.

4.1. Estaciones gemelas y espacio de acciones

La línea de Bosch está formada por pares de estaciones paralelas (y algunos tríos) que comparten predecesoras y sucesoras y se reparten la carga casi por igual; por ejemplo, en sus flujos principales las estaciones L0_S2 y L0_S3 reciben 339 345 y 334 054 casos. Identificamos las gemelas automáticamente a partir del solapamiento de sus conjuntos de predecesoras y sucesoras en los flujos principales: 29 de las 52 estaciones tienen una. Desviar una pieza de una estación a su gemela mantiene su trayectoria coherente con el grafo del proceso. La equivalencia estructural no garantiza un equipamiento o una capacidad equivalentes, que el conjunto de datos anonimizado no describe; una planta tendría que confirmar cada gemela. El espacio de acciones es Reroute(S, T, f), que desvía una fracción f de las piezas que entran en S a su gemela T; Restore(S); Maintenance(S, d), que repara S durante d minutos; y NoOp.

4.2. Validador simbólico

Toda acción, de cualquier planificador, se comprueba frente al grafo del proceso y a las intervenciones en curso antes de ejecutarse: las estaciones deben existir, el destino de un desvío debe ser una gemela estructural, los desvíos no deben encadenarse y una estación no puede repararse dos veces a la vez. El validador nunca juzga si una acción es una buena idea, solo si es admisible.

4.3. Modelo de efectos

La simulación está guiada por la traza: los instantes de inicio proceden de los datos registrados y no se desplazan; los fallos y las acciones actúan sobre las duraciones de las transiciones, y el tiempo de vida de una pieza es la suma de las duraciones de sus transiciones. Por tanto, el retraso no se propaga en forma de colas; el modelo estima el tiempo perdido por pieza. Una "transición" de Bosch es el tiempo entre dos marcas de estación y mide sobre todo espera: su mediana es 0 min y su percentil 90 es de 18–30 min, pero su percentil 99 llega a 4 100–5 900 min. Por ello los efectos son *aditivos* y proporcionales al tiempo de proceso de cada estación τ_S (la mediana de sus transiciones de hasta 60 min), nunca multiplicativos sobre una transición que incluye días de almacenamiento. τ_S vale 6 o 12 min en las ocho estaciones donde se inyectan fallos, idéntico en todas las trazas. El modelo de efectos es:

- **Fallo** de factor k en S: $+(k(t)-1)\,\tau_S$ en cada transición que sale de S, donde $k(t)$ es un *escalón* (k desde t_0), *intermitente* (k durante 60 min, 1 durante 120 min, y así sucesivamente) o una *deriva* (crece linealmente de 1 a k en 600 min).

- **Desvío**: una fracción determinista de las piezas (seleccionadas mediante un hash del identificador del caso) va a T, que añade entonces $\alpha\, f\, \tau_T$ por la carga extra.

- **Mantenimiento**: los fallos de S ya activos cuando empieza la reparación se eliminan cuando termina; un fallo que empieza durante una reparación no se elimina. Durante la reparación, en modo *contingencia* (por defecto) una capacidad de respaldo fuera del modelo procesa las piezas, añadiendo $0{,}5\,\tau_S$; en modo *parada* las piezas esperan a que termine la reparación. Reparar una estación sana cuesta y no arregla nada, y eso es lo que encarece las falsas alarmas.

Como el modelo es aditivo, el retraso en exceso de una ejecución se descompone exactamente en los minutos añadidos a cada transición, lo que permite medirlo en cualquier ventana temporal. El conjunto de datos está anonimizado y no proporciona capacidades, tiempos de reparación ni fallos etiquetados, de modo que todos los fallos son sintéticos y todo parámetro que no pudimos fundamentar en los datos (severidad k, tiempo de reparación d, congestión α, modo de mantenimiento) se varía en un análisis de sensibilidad (sección 7.2).

4.4. Una capa Analyze por estación sobre los tiempos de proceso

La sección 7.1 muestra que el detector heredado informa de los fallos inyectados pocas veces y solo dentro de ráfagas de toda la línea. Su test por estación compara medias de transiciones completas, dominadas por horas o días de almacenamiento, de modo que un fallo que añade minutos de proceso apenas las mueve. Por ello añadimos una capa Analyze alternativa que mira solo el régimen de proceso: las transiciones que salen de una estación y duran como máximo 60 min. Para cada estación, de forma causal, toma como referencia el percentil 75 de esas transiciones en las 24 h anteriores y la tasa p_0 con la que lo superan (con suavizado de Laplace, porque las marcas de tiempo tienen una granularidad de 6 min); entre las últimas N transiciones de proceso de la estación, cuenta las que superan la referencia y marca la estación cuando ese recuento es improbable bajo una Binomial(N, p_0) al nivel α. No se marca nada durante un calentamiento de 8 h; las estaciones marcadas en un mismo lote de eventos simultáneos se comunican en una sola alarma, y una estación no vuelve a marcarse dentro de un periodo refractario. N, α y el periodo refractario se eligieron en la traza de desarrollo (sección 6): $N = 10$, $\alpha = 10^{-3}$ y 6 min. Con esta capa, las alarmas de estación heredadas ya no se pasan a los planificadores. Los planificadores no cambian.

5. Planificadores

Todos los planificadores reciben, para cada alarma, la alarma y el estado de sus propias intervenciones anteriores. Ninguno ve los fallos inyectados. Todos responden del mismo modo ante una estación que consideran averiada: desviar a una gemela libre si existe y, después, reparar. Todos los planificadores, incluidos los LLM, dejan de actuar 240 min sobre una estación en la que acaban de actuar.

5.1. Planificadores basados en reglas

- **Ingenuo**: actúa sobre toda estación nombrada en una alarma.
- **Calibrado**: actúa cuando el número de alarmas de una estación en una ventana de 120 min es improbable (cola de Poisson, $\alpha = 10^{-3}$, al menos 3 alarmas) bajo su tasa base, aprendida en una traza sin fallos *distinta*.
- **Adaptativo**: el mismo test, frente a la tasa observada en las 24 h anteriores de la misma traza, de forma causal, tras un calentamiento de 8 h.

- **Adaptativo + veto por carga** (control): la regla adaptativa, ignorando las estaciones que reciben carga que el propio controlador desvió hacia ellas.
- **Solo gemelas** (control): actúa sobre toda estación marcada que tenga una gemela libre e ignora las demás.

Los dos controles no son métodos propuestos; implementan como reglas simples comportamientos observados en planificadores LLM, de modo que la contribución de un LLM se mide más allá de ellos. La ventana, el nivel y la historia por defecto se fijaron antes de cualquier evaluación; además los reajustamos en la traza de desarrollo (sección 6).

5.2. Planificadores LLM

Para cada alarma con estaciones accionables, el LLM debe devolver, para cada una de ellas, un veredicto (*degradada*, *normal*, *autoinducida* o *evidencia insuficiente*) seguido de un tipo de acción (ninguna, reparar, o desviar y reparar). La salida está restringida por un esquema JSON cuyos nombres de estación se limitan a las accionables, de modo que el decodificador no puede inventar una estación. El código elige la gemela, descarta toda acción incoherente con el propio veredicto del modelo y pasa el resto al validador.

- **C1** recibe la alarma, las estaciones accionables, si cada una tiene una gemela libre y las intervenciones en curso: la misma información estructural que las reglas.
- **C2 (RAG)** recibe además conocimiento recuperado de la memoria operativa del controlador: para cada estación, las alarmas observadas en las últimas 2 y 24 h frente al número que predice su tasa base sin fallos, si el controlador le está desviando carga, si sus alarmas bajaron tras reparaciones anteriores y sus registros más recientes. La recuperación se indexa por estación y se ordena por recencia; todas las estadísticas las calcula el código.
- **Híbrido**: el test adaptativo de detección de cambios propone las estaciones; el LLM, con el conocimiento de C2, confirma o veta y elige la acción. Una estación vetada no se vuelve a examinar durante 120 min. El prompt indica al modelo que el test suele acertar pero no ve el contexto, y da como ejemplos la carga que el controlador desvió hacia una estación, una reparación anterior que no redujo sus alarmas y *"a burst of alarms shared by many stations at once"* (una ráfaga de alarmas compartida por muchas estaciones a la vez).

El prompt de sistema describe cómo la capa Analyze emite las alarmas. La versión v2 del prompt describe el detector heredado; la v3, usada con el detector por estación, solo difiere de la v2 en ese párrafo. Ambas se fijaron en la traza de desarrollo antes de cualquier ejecución en las trazas reservadas.

Modelos. Usamos Qwen2.5-7B-Instruct [17], cuantizado a 4 bits y servido localmente con Ollama [18] a temperatura 0; y Claude Fable 5.1 (Anthropic), accedido mediante el Claude Agent SDK [19] con el esfuerzo en *low*, sin herramientas y sin fuentes de contexto externas. El modelo de frontera no expone un ajuste de temperatura, por lo que sus ejecuciones no son reproducibles bit a bit. Todas las peticiones y respuestas de ambos modelos se almacenan, indexadas por la petición completa. Las peticiones idénticas son frecuentes, porque antes del inicio del fallo un escenario produce los mismos estados que la ejecución sin fallo de su traza, y se sirven desde ese almacén: con el detector por estación, 2 403 de las 26 933 llamadas de C2 sobre Fable 5.1 fueron nuevas. Una llamada nueva tarda una mediana de 5–8 s con cualquiera de los dos modelos, poco frente a fallos que cuestan horas. Cada decisión es una única muestra del modelo; no estimamos la variabilidad entre ejecuciones. Fable 5.1 se usó a través de una suscripción de Claude, con el identificador de modelo `claude-fable-5-1`, entre el 6 y el 15 de octubre de 2026. Un primer piloto de desarrollo se descartó porque el SDK pasaba al modelo las definiciones de herramientas de los conectores de la cuenta; después se restringió el entorno a ninguna herramienta y se añadió su identificador a la clave del almacén, de modo que no se reutilizó ninguna respuesta de ese piloto. Los prompts figuran en el apéndice Apéndice A.

6. Protocolo experimental

Escenarios. Se inyecta un fallo de factor $k = 3$ en una de ocho estaciones, cuatro con gemela (L0_S2, L0_S4, L0_S6, L3_S35) y cuatro sin ella (L0_S1, L0_S8, L3_S29, L3_S33), todas con tiempos de proceso observables en cada traza, con una de las tres formas de fallo y en uno de tres inicios por estación y forma. Los inicios se sortean uniformemente entre 24 h después del primer evento de la traza y 24 h antes del último, con un generador sembrado con el nombre del escenario. Cada ejecución se detiene 24 h después del inicio del fallo. Cada traza tiene además una ejecución sin fallos sobre toda su longitud. Las reparaciones duran 60 min, $\alpha = 0{,}5$, y el mantenimiento funciona en modo contingencia salvo que se indique lo contrario.

Trazas de desarrollo y reservadas. La primera traza (0–100m) es la de desarrollo; las otras tres están reservadas (216 escenarios con fallo). Solo en la traza de desarrollo elegimos los parámetros del detector por estación (27 configuraciones, seleccionadas por la recuperación de la regla adaptativa con veto por carga), reajustamos las reglas (45 configuraciones de ventana, nivel e historia para cada regla adaptativa, 15 de ventana y nivel para la calibrada) y fijamos los prompts. Cada versión de prompt probada y sus resultados quedan registrados en el repositorio. Las reglas calibradas y C2 aprenden sus tasas base en una traza sin fallos distinta de aquella en la que se evalúan (100m–200m a partir de la traza de desarrollo; 200m–300m y 500m–600m, una a partir de la otra). Durante el desarrollo de los prompts, las ejecuciones de desarrollo tomaron sus tasas base de 100m–200m, que después queda reservada; esto no expone ninguna etiqueta de fallo.

Métricas. Cada ejecución reproduce la misma traza, de modo que cada pieza se compara consigo misma en una ejecución de referencia sin fallos. La *recuperación a 24 h* de un planificador sobre un conjunto de escenarios es $1-\sum E_{24}(\text{planificador})/\sum E_{24}(\text{no hacer nada})$, donde E_{24} es el retraso añadido a las transiciones en las 24 h siguientes al inicio del fallo: 1 elimina el coste de los fallos; 0 no es mejor que no hacer nada; los valores negativos significan que las propias acciones del planificador cuestan más que los fallos. Un planificador *actúa* en un escenario si repara la estación averiada en esas 24 h; las *reparaciones falsas* son reparaciones de otras estaciones en la misma ventana. El *coste de las falsas alarmas* es el retraso en exceso por hora en las ejecuciones sin fallos.

Estadística. El estimando principal es la diferencia de recuperación a 24 h entre dos planificadores, con un intervalo bootstrap al 95 % [20] que remuestrea los escenarios dentro de su traza (10 000 remuestreos). Como los escenarios de una misma traza comparten el ruido de alarmas y tres trazas son pocos grupos, presentamos también un bootstrap en dos niveles que remuestrea trazas y, dentro de ellas, escenarios, y la diferencia dejando fuera cada traza. Como análisis secundarios: la prueba de rangos con signo de Wilcoxon sobre el retraso por escenario [21], que trata los escenarios como independientes y es, por tanto, condicional a estas trazas, con ajuste de Holm [22] sobre las filas de cada bloque de tabla, y la correlación biserial de rangos para muestras pareadas r como tamaño del efecto. El desglose por forma de fallo es exploratorio.

Tabla 1: Lo que cada capa Analyze informa sobre la estación averiada en las 24 h siguientes al inicio del fallo, sin planificador, en los 216 escenarios con fallo reservados, y en las mismas ventanas sin el fallo.

Capa Analyze	Escenarios con alguna alarma (con / sin fallo)	Alarmas por escenario (con / sin fallo)	Estaciones por alarma
Detector heredado	108 / 52	5,4 / 1,2	6,5
Detector por estación	122 / 107	15,1 / 3,6	2,3

7. Resultados

7.1. La capa Analyze heredada da demasiado poca evidencia

El detector de estaciones heredado emite sus alarmas en ráfagas que siguen los lotes de producción, no los fallos. Sin fallos, en la traza reservada 100m–200m emite 63, 18, 0, 0, 0, 2, 0 y 1 alarmas en bloques sucesivos de 12 h; en 200m–300m, 66 en las primeras 12 h, ninguna en las 60 h siguientes y después 25, 18 y 16; en la traza de desarrollo, 30, 9 y luego ninguna. La tabla 1 muestra lo que informa en las 24 h siguientes a un fallo, sin planificador. Nombra a la estación averiada en la mitad de los escenarios reservados, 5,4 veces de media frente a 1,2 en la misma ventana sin el fallo, y cada una de esas alarmas incluye 6,5 estaciones de media, porque el detector solo se dispara cuando más del 10 % de las estaciones se marcan en un mismo lote. Compara tiempos medios de transición dominados por el almacenamiento, de modo que un fallo que añade 12 min de proceso apenas los mueve.

Con esta capa Analyze, ningún planificador recupera retraso en 24 h (tabla 3, bloque superior): las reglas van de $-0{,}03$ a $-0{,}02$, C2 sobre Fable 5.1 llega a $+0{,}07$ $[-0{,}00; +0{,}14]$ y el híbrido sobre Fable 5.1, que veta casi todos los candidatos, a $+0{,}03$. Los planificadores tienen demasiado poco sobre lo que actuar.

7.2. Una capa Analyze por estación

El detector por estación es más ruidoso en términos absolutos (tabla 1): nombra a una estación sin fallo en la mitad de las ventanas. Pero informa de la estación averiada tres veces más que el detector heredado, 15,1 alarmas por escenario frente a 3,6 sin el fallo, y en alarmas que incluyen 2,3 estaciones de media, de modo que un test de tasa sobre sus alarmas puede aislar la estación. En la traza de desarrollo, todas las configuraciones del detector por estación nombran a la estación averiada en 24 h en los 72 escenarios de desarrollo; la elegida permite a la regla adaptativa con veto por carga recuperar $+0{,}84$ allí. La tabla 2 recoge los planificadores basados en reglas con este detector en las trazas reservadas y un análisis de sensibilidad de

Tabla 2: Planificadores basados en reglas con el detector por estación, trazas reservadas (216 escenarios con fallo): recuperación a 24 h. Punto central: $k = 3$, reparaciones de 60 min, $\alpha = 0{,}5$, modo contingencia. Filas de sensibilidad: todas las formas de fallo, un parámetro cambiado. Hiperparámetros por defecto.

	Ingenuo	Calibrado	Adaptativo	Solo gemelas
Todas las formas	−0,28	+0,13	+0,17	−0,20
escalón	−0,07	+0,23	+0,23	−0,07
intermitente	−1,12	−0,29	−0,27	−0,64
deriva	−0,10	+0,22	+0,30	−0,13
$\alpha = 0$	–	+0,37	+0,41	+0,22
$\alpha = 1$	–	−0,07	−0,03	−0,61
$k = 1{,}5$	–	−1,51	−1,47	−1,75
$k = 5$	–	+0,37	+0,41	+0,03
Reparación de 15 min	–	+0,54	+0,58	+0,20
Reparación de 240 min	–	−1,22	−1,22	−1,73
Modo parada	–	−2,48	−2,55	−0,81

un factor cada vez sobre todas las formas de fallo. Actuar sobre las alarmas del detector compensa solo cuando los fallos son graves en relación con el coste de una reparación y una capacidad de respaldo absorbe el trabajo durante las reparaciones. Reducir a la mitad el factor del fallo ($k = 1{,}5$), cuadruplicar el tiempo de reparación (240 min) o parar la línea durante las reparaciones hace que todas las reglas sean mucho peores que no hacer nada, porque las reparaciones falsas cuestan entonces más que los fallos que arreglan; la congestión completa ($\alpha = 1$) anula el beneficio. Reparaciones más cortas, fallos más graves o la ausencia de congestión lo multiplican por más de dos. El planificador ingenuo, que actúa sobre toda alarma, es peor que no hacer nada en el punto central, y los fallos intermitentes derrotan allí a todas las reglas. Este análisis cubre solo las reglas; no lo repetimos para los planificadores LLM.

7.3. Comparación en las trazas reservadas

La tabla 3 y la figura 2 recogen todos los planificadores en las tres trazas reservadas. Con el detector por estación, las reglas recuperan entre el 13 y el 20 % con su configuración por defecto; reajustarlas en la traza de desarrollo ayuda a la regla calibrada (+0,27) pero no a las adaptativas (+0,14 y +0,18). El planificador ingenuo es peor que no hacer nada con los dos detectores (−0,25 y −0,28). La traza de desarrollo es bastante más fácil que las reservadas (+0,84 frente a +0,20 para la misma regla).

Tabla 3: Trazas reservadas (216 escenarios con fallo, 3 ejecuciones sin fallos), horizonte de 24 h. Recuperación: con intervalo bootstrap al 95 % estratificado por traza. Actúa: escenarios en los que se repara la estación averiada en 24 h. Reparaciones falsas: reparaciones de otras estaciones en 24 h, por escenario. Coste de falsas alarmas: min-pieza/h en las ejecuciones sin fallos, media de las tres trazas.

Planificador	Modelo	Recuperación	Escalón	Interm.	Deriva	Actúa	Reparaciones falsas	Coste de falsas alarmas
Detector heredado								
Regla calibrada	–	−0,03 [−0,12; +0,04]	+0,04	−0,38	+0,04	62	4,4	108
Regla adaptativa	–	−0,03 [−0,14; +0,06]	+0,06	−0,44	+0,06	77	6,2	67
Adaptativa + veto por carga	–	−0,02 [−0,12; +0,07]	+0,08	−0,40	+0,06	75	5,9	64
C2 (RAG)	Qwen 7B	−0,15 [−0,27; −0,05]	−0,06	−0,73	+0,02	104	9,5	159
Híbrido	Qwen 7B	−0,04 [−0,10; +0,03]	−0,02	−0,25	+0,05	51	4,0	38
C2 (RAG)	Fable 5.1	+0,07 [−0,00; +0,14]	+0,13	−0,17	+0,12	54	3,9	58
Híbrido	Fable 5.1	+0,03 [−0,01; +0,08]	+0,09	−0,03	+0,00	24	0,9	7
Detector por estación								
Regla calibrada	–	+0,13 [+0,01; +0,24]	+0,23	−0,29	+0,22	101	5,5	107
Regla adaptativa	–	+0,17 [+0,05; +0,27]	+0,23	−0,27	+0,30	108	5,9	98
Adaptativa + veto por carga	–	+0,20 [+0,09; +0,30]	+0,26	−0,20	+0,33	108	5,5	89
Solo gemelas (control)	–	−0,20 [−0,30; −0,11]	−0,07	−0,64	−0,13	60	6,2	108
C1	Qwen 7B	−0,05 [−0,17; +0,06]	+0,06	−0,55	+0,06	90	7,6	121
C2 (RAG)	Qwen 7B	−0,25 [−0,42; −0,10]	−0,05	−1,06	−0,08	117	14,0	203
Híbrido	Qwen 7B	+0,29 [+0,18; +0,38]	+0,32	−0,03	+0,41	106	5,2	74
C1	Fable 5.1	+0,08 [+0,04; +0,14]	+0,02	+0,10	+0,14	19	0,1	1
C2 (RAG)	Fable 5.1	+0,14 [+0,02; +0,24]	+0,23	−0,25	+0,22	105	7,1	100
Híbrido	Fable 5.1	**+0,41** [+0,32; +0,50]	+0,35	+0,29	+0,53	87	**1,8**	**25**

Con el detector por estación, los planificadores se separan. El híbrido sobre Fable 5.1 llega a +0,41, la recuperación más alta, reparando 1,8 estaciones sanas por escenario y con un coste de 25 min-pieza/h sin fallos, frente a 5,5 y 89 de la regla adaptativa con veto por carga. Es el único planificador con una recuperación claramente positiva en los fallos intermitentes (+0,29). Actúa sobre la estación averiada en menos escenarios que las reglas (87 frente a 108 de 216), pero pierde mucho menos en reparaciones falsas. El mismo híbrido sobre el modelo de 7B llega a +0,29. Usados sin el test, los planificadores LLM lo hacen peor: C2 sobre Fable 5.1 llega a +0,14, por debajo de la regla, y C2 sobre el modelo de 7B a −0,25, con 14 reparaciones falsas por escenario. Sin conocimiento recuperado, Fable 5.1 (C1) actúa sobre la estación averiada solo en 19 escenarios y casi nunca sobre una sana; el modelo de 7B actúa igualmente. La figura 3 sitúa cada planificador en el compromiso entre la recuperación ante fallos y el coste de actuar sobre el ruido.

7.4. Comparación estadística

La tabla 4 compara el híbrido sobre Fable 5.1 con los demás planificadores con el detector por estación. Frente a la regla adaptativa con veto por carga, su recuperación es mayor en +0,21 [+0,14; +0,28]; pierde menos tiempo de producción en 139 escenarios y más en 18 ($r = 0{,}78$). La ventaja

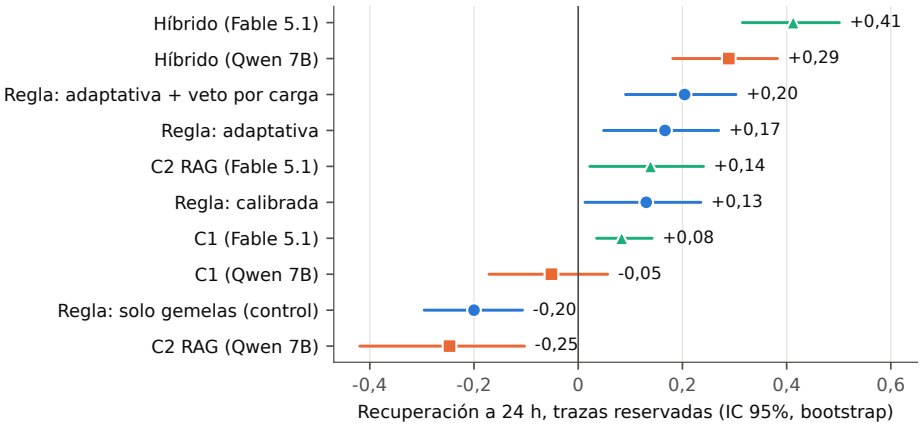


Figura 2: Recuperación a 24 h en las trazas reservadas con el detector por estación. Círculos: planificadores basados en reglas; cuadrados: Qwen2.5 7B; triángulos: Claude Fable 5.1.

se mantiene en cada traza reservada (+0,23, +0,15 y +0,22), dejando fuera cualquier traza (de +0,19 a +0,23) y al remuestrear las trazas como grupos ([+0,11; +0,28]; positiva en las 27 extracciones ordenadas de tres trazas). También se mantiene frente a las reglas reajustadas en la traza de desarrollo (+0,15 [+0,08; +0,22] frente a la mejor de ellas) e incluso frente a la configuración de la rejilla que es la mejor en las propias trazas reservadas (+0,13 [+0,06; +0,21]), que ningún protocolo honesto podría seleccionar. Es mayor en los fallos intermitentes (+0,49) y de deriva (+0,20) que en los de escalón (+0,09, intervalo que incluye el cero). El híbrido sobre el modelo de 7B va por delante de la regla por defecto (+0,09, [+0,05; +0,13] al remuestrear trazas), pero empata con la mejor regla reajustada (+0,02 [−0,02; +0,07]). C2 sobre Fable 5.1 queda por detrás de la regla (−0,07 [−0,17; −0,00] al remuestrear trazas).

7.5. En qué se apoyan los vetos

El mismo híbrido sobre Fable 5.1 es el planificador más conservador con el detector heredado y el mejor con el detector por estación, con el mismo prompt. Las razones que da apenas cambian. Codificados con una regla explícita de palabras clave (*burst*, *line-wide*, *common cause*, *systemic*; la regla y los recuentos están en el repositorio), el 99 % de sus vetos argumenta que las alarmas llegan en ráfagas compartidas por muchas estaciones con el detector heredado (1 596 de las 1 607 llamadas con veto), y el 91 % con el

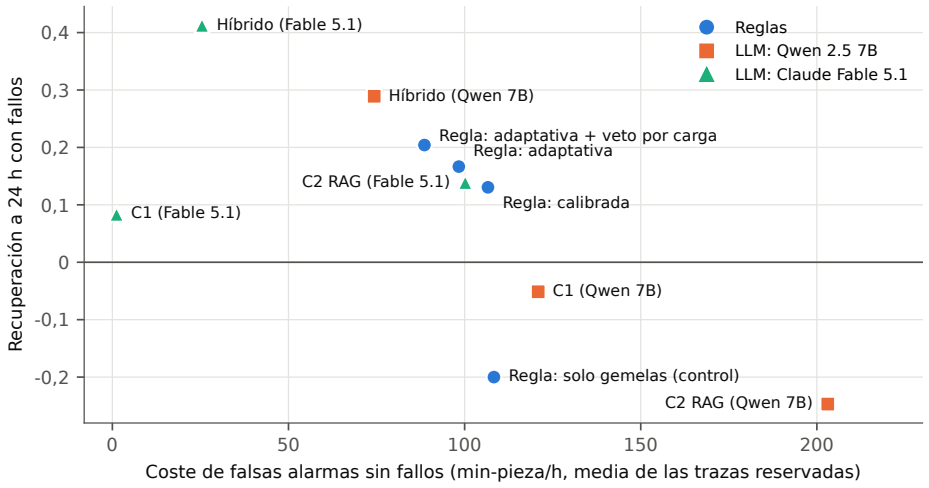


Figura 3: Recuperación a 24 h con fallos frente al coste de las falsas alarmas sin fallos, trazas reservadas, detector por estación.

detector por estación (2 587 de 2 833); contados sobre respuestas distintas y no sobre llamadas, el 97 % (119 de 123) y el 87 % (249 de 285). Sus vetos de la estación averiada citan esta razón en 148 de 148 y en 139 de 142 casos. El detector heredado emite una alarma *solo* cuando más del 10 % de las estaciones se marcan en el mismo lote, de modo que con él toda alarma es una ráfaga compartida y el argumento lo veta casi todo: el híbrido vetó al menos una estación en 1 607 de sus 1 812 llamadas. Con el detector por estación, una ráfaga es un patrón posible entre otros, y el mismo argumento separa las perturbaciones de toda la línea de los fallos aislados: el híbrido veta en 2 833 de 4 348 llamadas y recupera el doble que la regla. El prompt del híbrido menciona esas ráfagas entre las razones para dudar del test; el modelo de 7B, con el mismo prompt, usa este argumento en menos del 1 % de sus vetos con cualquiera de los dos detectores. Las razones declaradas no bastan para mostrar cómo pondera el modelo la evidencia, pero el contraste entre las dos capas Analyze, con el planificador y el prompt fijos, muestra que lo que codifican las alarmas decide si el veto ayuda o perjudica.

7.6. Por qué importa el diseño de evaluación

En una primera fase de esta investigación, cada fallo se inyectaba 600 min después del primer evento de una traza, una reparación eliminaba cualquier fallo que empezara durante ella y la recuperación se medía sobre toda la tra-

Tabla 4: El híbrido sobre Claude Fable 5.1 frente a otros planificadores, detector por estación, pareado sobre los 216 escenarios con fallo reservados. Valores p de Wilcoxon ajustados por Holm sobre las filas de cada bloque; r: correlación biserial de rangos para muestras pareadas. Δ: diferencia de recuperación a 24 h (positiva a favor del híbrido sobre Fable 5.1), intervalo bootstrap al 95 % estratificado por traza. Reglas reajustadas: elegidas en la traza de desarrollo, salvo la última fila (la mejor en las trazas reservadas). El desglose por forma de fallo es exploratorio.

Frente a	Mejor / peor	p (Holm)	r	Δ recuperación [IC 95 %]
Adaptativa + veto por carga	139 / 18	$< 10^{-15}$	0,78	+0,21 [+0,14; +0,28]
Regla adaptativa	144 / 14	$< 10^{-15}$	0,81	+0,25 [+0,17; +0,32]
Regla calibrada	138 / 22	$< 10^{-15}$	0,77	+0,28 [+0,20; +0,37]
Híbrido, Qwen 7B	133 / 25	$< 10^{-12}$	0,68	+0,12 [+0,06; +0,19]
C2 (RAG), Fable 5.1	141 / 20	$< 10^{-15}$	0,82	+0,27 [+0,20; +0,35]
Frente a reglas reajustadas				
Calibrada, reajustada	116 / 37	$< 10^{-10}$	0,62	+0,15 [+0,08; +0,22]
Adaptativa + veto, reajustada	143 / 18	$< 10^{-15}$	0,78	+0,23 [+0,16; +0,31]
Adaptativa + veto, mejor en reservadas	128 / 31	$< 10^{-12}$	0,66	+0,13 [+0,06; +0,21]
Frente a adaptativa + veto por carga, por forma de fallo				
escalón	42 / 7	$< 10^{-4}$	0,64	+0,09 [−0,02; +0,22]
intermitente	49 / 5	$< 10^{-6}$	0,85	+0,49 [+0,35; +0,64]
deriva	48 / 6	$< 10^{-6}$	0,83	+0,20 [+0,13; +0,27]

za. Con ese diseño el detector heredado parecía funcionar: las reglas adaptativas recuperaban el 52–54 %, C2 sobre Fable 5.1 el 58 % y el híbrido sobre Fable 5.1 el 24 % (tabla 5). Esas ganancias eran un artefacto del diseño. El inicio fijo caía dentro de la ráfaga de alarmas de arranque del detector y justo después del calentamiento de 8 h de la memoria de los planificadores, cuando C2 repara muchas estaciones a la vez; las reparaciones rutinarias coincidían entonces con el fallo o lo precedían por minutos y, con la regla de reparación anterior, lo eliminaban sin coste. Sacar los inicios del periodo de arranque y permitir que las reparaciones eliminen solo los fallos activos hizo desaparecer esas ganancias e invirtió el orden de los dos diseños LLM. Presentamos aquí el primer diseño porque el error es fácil de cometer: un controlador en bucle cerrado evaluado en un único instante de fallo fijo hereda lo que le esté ocurriendo a la línea en ese instante.

8. Discusión

La capa Analyze decide. Cerrar el bucle original [3] sobre su propio detector no recupera nada en un día, sea cual sea el planificador: ninguno se aleja de no hacer nada en más de 0,07, porque el detector da demasiado poca evidencia, y demasiado poco específica, sobre la estación averiada. Sustituirlo por un test por estación sobre los tiempos de proceso, elegido en una traza de

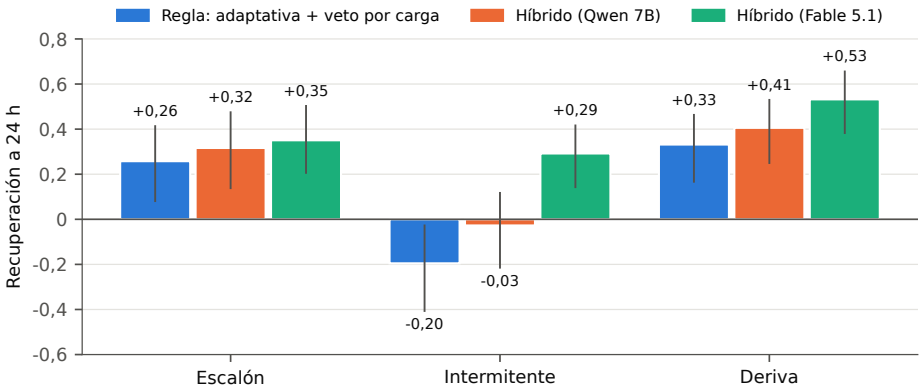


Figura 4: Recuperación a 24 h por forma de fallo en las trazas reservadas, detector por estación (72 escenarios por forma). Bigotes: intervalo bootstrap al 95 % estratificado por traza.

Tabla 5: Los mismos planificadores con el primer diseño de evaluación (v1: fallo a los 600 min, una reparación elimina los fallos que empiezan durante ella, recuperación sobre toda la traza, 72 escenarios) y con el actual (v2: tres inicios aleatorios tras el periodo de arranque, una reparación elimina solo los fallos activos, recuperación a 24 h, 216 escenarios). Trazas reservadas.

Planificador	v1, heredado	v2, heredado	v2, por estación
Adaptativa + veto por carga	+0,54	−0,02	+0,20
C2 (RAG), Qwen 7B	+0,20	−0,15	−0,25
Híbrido, Qwen 7B	+0,41	−0,04	+0,29
C2 (RAG), Fable 5.1	+0,58	+0,07	+0,14
Híbrido, Fable 5.1	+0,24	+0,03	+0,41

desarrollo y mantenido fijo, lleva a la mejor regla de −0,02 a +0,20 y al mejor planificador de +0,07 a +0,41. En términos de MAPE-K, la información que pasa de Analyze a Plan es lo primero que hay que hacer bien.

Proponer y vetar. Con alarmas útiles, el LLM aporta más como veto sobre un test estadístico que como triaje por sí solo. El test solo propone un candidato cuando la tasa de alarmas de una estación se aparta de su propia historia; el modelo, con las estadísticas recuperadas, rechaza los candidatos que explican una perturbación de toda la línea, un desvío o una reparación anterior ineficaz. Este reparto de funciones mantiene la decisión anclada en un test transparente y usa el modelo donde tiene ventaja: ponderar varias piezas de contexto que una regla tendría que enumerar. Por sí solos, los dos

planificadores LLM lo hacen peor que la regla; el modelo de frontera sin conocimiento recuperado casi nunca actúa.

La capacidad importa dentro de la arquitectura adecuada. El mismo híbrido recupera +0,41 sobre Fable 5.1 y +0,29 sobre el modelo de 7B, que solo empata con la mejor regla reajustada. Es un contraste entre dos sistemas, que difieren en tamaño, cuantización, proveedor y condiciones de inferencia, no una curva de capacidad medida.

Implicaciones para la práctica. Dentro de los límites de un estudio con fallos sintéticos, cuatro ideas son trasladables a arquitectos de automatización e ingenieros de mantenimiento. Primero, conectar alarmas por umbral directamente a órdenes de mantenimiento es perjudicial: el planificador ingenuo es peor que no hacer nada con los dos detectores, porque las reparaciones falsas se acumulan; las alarmas deben racionalizarse antes de que disparen acciones, por analogía con lo que la práctica de gestión de alarmas exige para los operadores [13]. Segundo, un detector debería comparar cada estación con su propio comportamiento reciente en la parte de la señal que un fallo afecta, aquí los tiempos de proceso y no transiciones dominadas por el almacenamiento. Tercero, un test de cambio de tasa es una capa de propuesta barata y transparente, y la referencia con la que debería compararse cualquier controlador aprendido o basado en LLM. Cuarto, un LLM puede situarse sobre ese test como veto asesor; su no determinismo y su sensibilidad a cómo se le presentan las alarmas aconsejan mantener toda acción detrás de un validador simbólico, como aquí, y, en una planta, detrás de la aprobación de una persona.

Los conjuntos de desarrollo engañan. El detector por estación y las reglas reajustadas se eligieron en la traza de desarrollo, donde la regla adaptativa recupera +0,84; en las trazas reservadas recupera +0,20, y las configuraciones reajustadas allí ayudan a una regla y no a las otras. Con el primer diseño, tres conclusiones extraídas de la traza de desarrollo también fueron erróneas. Una sola traza es demasiado pequeña para seleccionar diseños, y los controladores basados en LLM deberían evaluarse con datos que no se usaron para escribir sus prompts, con inicios de fallo que no coincidan con el arranque del controlador.

Amenazas a la validez. De constructo: los fallos son sintéticos y el modelo de efectos es aditivo y guiado por la traza, sin colas, de modo que toda recuperación es una propiedad del modelo, no una propiedad medida de la línea de Bosch. Sus parámetros no verificables se barrieron de uno en uno para

las reglas (tabla 2): los fallos leves, las reparaciones largas, la congestión completa y el modo parada eliminan o invierten el beneficio de actuar, de modo que toda recuperación que presentamos vale solo para fallos graves, reparaciones cortas y capacidad de respaldo durante las reparaciones. No se hizo análisis de sensibilidad para los planificadores LLM. La recuperación se mide en tiempo de producción en las 24 h siguientes al fallo, sin ponderación económica; la ventana de 24 h puede caer en parte sobre horas sin producción, porque las trazas se registran por lotes. Las gemelas son estructurales, no equipos cuya equivalencia se haya verificado. *Interna*: cada decisión del modelo de frontera es una única muestra almacenada con esfuerzo bajo, y la mayoría de sus llamadas reutilizan una respuesta almacenada; los prompts forman parte del tratamiento, y el prompt del híbrido contiene la pista de las ráfagas que citan sus vetos; las reglas de control cubren el veto por carga, pero no todas las estadísticas que recupera C2, como el resultado de las reparaciones anteriores; el modelo de 7B está cuantizado a 4 bits. El detector por estación y el diseño de evaluación se introdujeron después de haber visto resultados con el primer diseño; ambos se fijaron en la traza de desarrollo, y el cambio quedó registrado en el repositorio antes de cualquier ejecución en las trazas reservadas con ellos. El modelo de frontera puede haber visto nuestros artículos anteriores [3, 4], que nombran la línea de Bosch, pero los fallos son sintéticos y no están descritos en ningún sitio que haya podido ver. *Externa*: una sola línea de producción y un solo conjunto de datos; dos modelos de lenguaje, sin tamaños intermedios. *Estadística*: los escenarios de una misma traza no son independientes; tres trazas reservadas son pocos grupos, algo que el remuestreo por trazas solo resuelve en parte.

9. Conclusión

Cerramos el bucle autonómico de una arquitectura de fabricación autocoordinada que, tal como se publicó, solo detectaba. Con su propio detector, ningún planificador recupera retraso: el detector emite ráfagas de alarmas de toda la línea e informa pocas veces de una estación averiada. Con un detector por estación sobre los tiempos de proceso, un test estadístico de cambio recupera alrededor de una quinta parte del retraso en un día, y un LLM de frontera con conocimiento operativo recuperado que veta los candidatos del test recupera alrededor de dos quintas partes, por delante de todas las configuraciones de reglas que probamos. Usado por sí solo, el mismo modelo no supera al test. Una evaluación anterior, con un instante de fallo fijo, había sugerido el orden contrario. El trabajo futuro validará el modelo de efectos

con datos de planta con capacidades y tiempos de reparación reales, ejecutará el híbrido sin la pista de las ráfagas y con muestras repetidas, añadirá tamaños de modelo intermedios y extenderá el análisis de sensibilidad a los planificadores LLM.

Apéndice A. Prompts

Prompt de sistema v3 de C1, tal como se envía a ambos modelos (exportado del código):

```
You are the Plan phase of an autonomic controller for a manufacturing line
     (the Bosch production line: about 50 stations on 4 sub-lines).

How alarms work. The Analyze phase checks each station separately. It
    raises an alarm listing the stations whose recent processing times (
    transitions of up to 60 minutes leaving the station) exceed the
    station's own recent history more often than chance would explain. In
    normal operation this detector also raises false alarms: a station
    named in an alarm may be perfectly healthy.

What you can do for each flagged station:
- "none": leave it alone;
- "maintain": repair it for 60 minutes. While it is being repaired a
    slower backup handles its work. Repairing a degraded station fixes it.
     Repairing a healthy station fixes nothing and only slows production;
- "reroute_and_maintain": only for a station with a free twin. The twin
    takes all of the station's load during the repair (the twin slows down
     because of the extra load), and the station is repaired.

Objective: minimise total production delay. Acting on a healthy station
    wastes production. Ignoring a truly degraded station lets its delay
    accumulate for as long as it stays degraded.

For EVERY station listed, first give a verdict, then an action:
- "degraded": the evidence shows the station itself is not working
    properly;
- "normal": the station is probably healthy and the alarm is a false alarm
    ;
- "self_inflicted": the station is alarming because of load you rerouted
    onto it;
- "insufficient_evidence": you cannot tell.
Only a "degraded" station may receive an action other than "none".

Answer only with JSON matching the schema. Keep "reason" under 30 words.
```

C2 añade:

```
You also receive RETRIEVED KNOWLEDGE for each station, computed from your
    operational log:
- how many alarms the station raised recently, and how many its normal (
    fault-free) alarm rate would predict over the same period;
- whether you are currently pushing extra load onto it;
- what happened to its alarms after you repaired it before;
- its recent log records.
How to read it:
- many more alarms than expected, sustained over the last 24 h, points to
    "degraded";
- about as many alarms as expected points to "normal";
- a station receiving load you rerouted onto it is "self_inflicted":
    repairing it does not help;
- if alarms did not drop after a previous repair, the alarms were probably
     not caused by a fault;
- with too little history, prefer "insufficient_evidence".
```

El híbrido añade, tras el texto de C2:

```
In this mode a statistical change detector has already screened the alarms
    . The stations listed were flagged because their recent alarm count is
     improbably high compared with their own recent history. The detector
    is usually right, but it cannot see context: load you rerouted onto a
    station, a previous repair that did not reduce its alarms, a burst of
    alarms shared by many stations at once. Give "degraded" unless the
    retrieved knowledge gives a concrete reason to doubt the detector;
    then give the verdict that reason supports.
```

El prompt v2, usado con el detector heredado, sustituye la primera frase del segundo párrafo del prompt de C1 (de “How alarms work” a “explain.”) por:

```
How alarms work. The Analyze phase raises an alarm listing stations whose
    recent transition times exceeded 1.3 times their reference time within
     the same batch of events.
```

El mensaje de usuario indica el instante, las estaciones que hay que juzgar con el estado de sus gemelas, las estaciones en reparación y los desvíos activos; para C2 y el híbrido añade, por estación, el conocimiento recuperado descrito en la sección 5. El esquema JSON de la respuesta está en el repositorio.

Disponibilidad de datos y código

El simulador sin interfaz gráfica, los planificadores, los prompts, el registro de desarrollo, cada decisión de los modelos con la razón que dieron, los resultados y los scripts que producen cada tabla y figura están disponibles en `https://github.com/mbsanchez/ssacsimulator`. El simulador usa solo la

biblioteca estándar de Python; la estadística y las figuras usan además SciPy y matplotlib. El conjunto de datos de Bosch es público [16]; los ficheros derivados de él (modelo de proceso y trazas de eventos) no se redistribuyen, y el repositorio incluye los scripts que los producen y sus sumas de verificación.

Declaración de contribución de autoría (CRediT)

Manuel Sánchez: conceptualización, metodología, investigación, visualización, redacción – borrador original. **Ernesto Exposito:** supervisión, redacción – revisión y edición.

Declaración de conflicto de intereses

Los autores declaran no tener intereses financieros en conflicto ni relaciones personales conocidas que pudieran haber influido en el trabajo presentado en este artículo.

Declaración sobre el uso de IA generativa y tecnologías asistidas por IA en la preparación del manuscrito

Durante la preparación de este trabajo los autores usaron Claude (Anthropic) para asistir en el desarrollo del software de simulación y análisis, redactar y revisar el texto del manuscrito, y como agente adversario que actuó como revisor para identificar debilidades y sugerir mejoras. El software fue revisado por el primer autor. Tras usar esta herramienta, los autores revisaron y editaron el contenido según fue necesario y asumen toda la responsabilidad sobre el contenido del artículo publicado. El uso de Qwen2.5 y Claude Fable 5.1 como planificadores forma parte del método estudiado en este artículo (sección 5) y no guarda relación con la preparación del manuscrito.

Referencias

[1] J. O. Kephart, D. M. Chess, The vision of autonomic computing, Computer 36 (1) (2003) 41–50. doi:10.1109/MC.2003.1160055.

[2] D. Weyns, An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective, Wiley-IEEE Press, 2021. doi:10.1002/9781119574910.

[3] M. Sánchez, E. Exposito, J. Aguilar, Implementing self-* autonomic properties in self-coordinated manufacturing processes for the Industry 4.0 context, Computers in Industry 121 (2020) 103247. doi:10.1016/j.compind.2020.103247.

[4] M. Sánchez, E. Exposito, J. Aguilar, Autonomic computing in manufacturing process coordination in Industry 4.0 context, Journal of Industrial Information Integration 19 (2020) 100159. doi:10.1016/j.jii.2020.100159.

[5] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, in: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275362.

[6] M. Li, Q. Zhou, W. Li, T. Qu, M. Yang, P. Jiang, A4PS: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing, Journal of Manufacturing Systems 85 (2026) 207–226. doi:10.1016/j.jmsy.2026.01.003.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[8] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Generative AI for self-adaptive systems: State of the art and research roadmap, ACM Transactions on Autonomous and Adaptive Systems 19 (3) (2024) 13:1–13:60. doi:10.1145/3686803.

[9] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Exploring the potential of large language models in self-adaptive systems, in: Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2024, pp. 77–83. doi:10.1145/3643915.3644088.

[10] N. Nascimento, P. Alencar, D. Cowan, Self-adaptive large language model (LLM)-based multiagent systems, in: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), 2023, pp. 104–109. doi:10.1109/ACSOS-C58168.2023.00048.

[11] L. Alves, G. L. B. De Oliveira, M. Chagas, A. Bandeira, P. H. Maia, C. Rocha, MAPER: Extending MAPE-K with LLM-based reasoning to manage unanticipated situations in self-adaptive systems, in: Proceedings of the 21st International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2026, pp. 239–250. doi:10.1145/3788550.3794886.

[12] M. Sánchez, E. Exposito, J. Aguilar, Industry 4.0: survey from a system integration perspective, International Journal of Computer Integrated Manufacturing 33 (10–11) (2020) 1017–1041. doi:10.1080/0951192X.2020.1775295.

[13] International Society of Automation, ANSI/ISA-18.2-2016: Management of alarm systems for the process industries, Tech. rep., International Society of Automation, Research Triangle Park, NC (2016).

[14] M. Basseville, I. V. Nikiforov, Detection of Abrupt Changes: Theory and Application, Prentice Hall, 1993.

[15] N. D. Thuan, L. H. Anh, Large language models for fault diagnosis in smart manufacturing: A comprehensive review, Intelligent Systems with Applications 31 (2026) 200714. doi:10.1016/j.iswa.2026.200714.

[16] Bosch, Bosch production line performance, Kaggle competition dataset, https://www.kaggle.com/c/bosch-production-line-performance (2016).

[17] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[18] Ollama, Ollama: run large language models locally, https://ollama.com, version 0.35.1 (2026).

[19] Anthropic, Claude agent sdk, https://code.claude.com/docs/en/agent-sdk, python package claude-agent-sdk 0.2.163 (2026).

[20] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[21] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[22] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.